Visitor

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

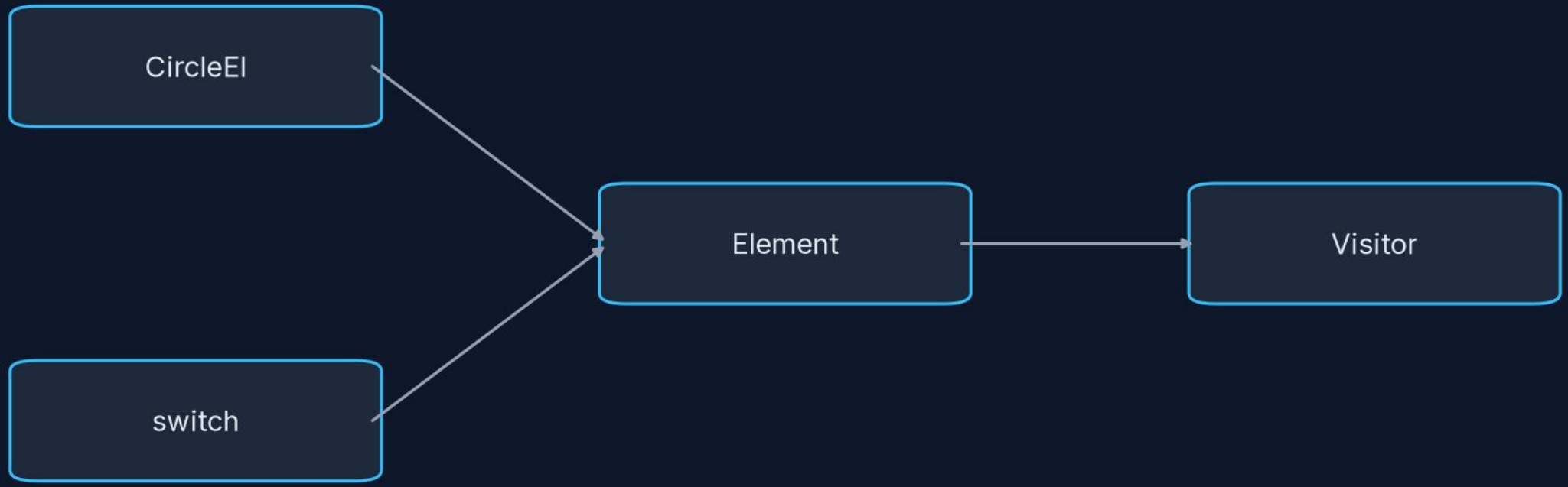
Today's Hook

- Today: Visitor as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Visitor structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Visitor
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/behavioral/visitor/ShapeVisitor.java
package com.buildwithwaqar.patterns.gof.behavioral.visitor;

sealed interface Element permits CircleEl, RectEl {
    <R> R accept(Visitor<R> visitor);
}

record CircleEl(double r) implements Element {
    public <R> R accept(Visitor<R> visitor) { return visitor.visit(this); }
}

record RectEl(double w, double h) implements Element {
    public <R> R accept(Visitor<R> visitor) { return visitor.visit(this); }
}

interface Visitor<R> {
    R visit(CircleEl c);
    R visit(RectEl r);
}

/** Visitor vs Java 25 pattern matching on sealed types. */
public final class ShapeVisitor {
    private ShapeVisitor() {}
```

Implementation (2)

```
    public static double area(Element e) {
        return switch (e) {
            case CircleEl(var r) -> Math.PI * r * r;
            case RectEl(var w, var h) -> w * h;
        };
    }

    public static double areaViaVisitor(Element e) {
        return e.accept(new Visitor<>() {
            public Double visit(CircleEl c) { return Math.PI * c.r() * c.r(); }
            public Double visit(RectEl r) { return r.w() * r.h(); }
        });
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.behavioral.visitor;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Visitor walkthrough ===");
    Element c = new CircleEl(2);
    Element r = new RectEl(3, 4);
    System.out.println("circleAreaPM=" + ShapeVisitor.area(c));
    System.out.println("circleAreaVisitor=" + ShapeVisitor.areaViaVisitor(c));
    System.out.println("rectArea=" + ShapeVisitor.area(r));
  }
}
```

Walkthrough Output

```
=== Visitor walkthrough ===
circleAreaPM=12.566370614359172
circleAreaVisitor=12.566370614359172
rectArea=12.0
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Visitor
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Visitor → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.